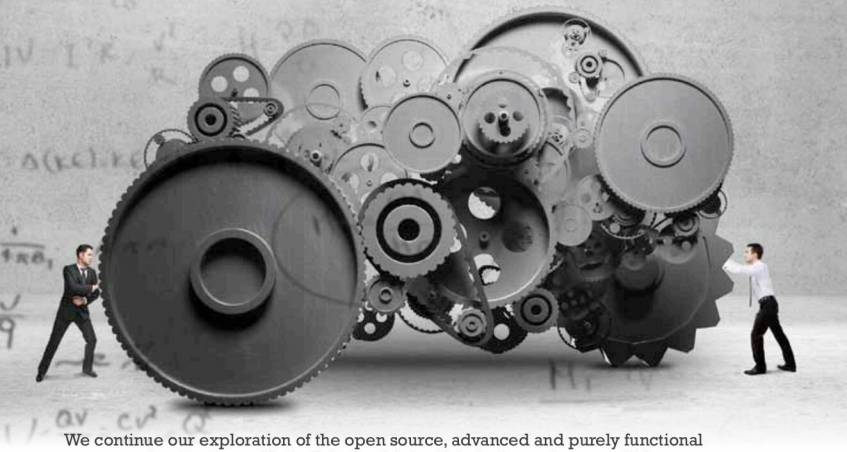
Experimenting with More Functions in Haskell



We continue our exploration of the open source, advanced and purely functional programming language, Haskell. In the third article in the series, we will focus on more Haskell functions, conditional constructs and their usage.

A function in Haskell has the function name followed by arguments. An infix operator function has operands on either side of it. A simple infix *add* operation is shown below:

```
*Main> 3 + 5
8
```

If you wish to convert an infix function to a prefix function, it must be enclosed within parentheses:

```
*Main> (+) 3 5
8
```

Similarly, if you wish to convert a prefix function into an infix function, you must enclose the function name within *backquotes(`)*. The *elem* function takes an element and a list, and returns true if the element is a member of the list:

```
*Main> 3 `elem` [1, 2, 3]
True
*Main> 4 `elem` [1, 2, 3]
False
```

Functions can also be partially applied in Haskell. A function that subtracts *ten* from a given number can be defined as:

```
diffTen :: Integer -> Integer
diffTen = (10 -)
```

Loading the file in GHCi and passing *three* as an argument yields:

```
*Main> diffTen 3
7
```

Haskell exhibits polymorphism. A type variable in a function is said to be polymorphic if it can take any type. Consider the last function that returns the last element in an array. Its type signature is: